

Guía de Consulta Rápida: Java, Estructuras y Matrices

1. Fundamentos y Características de Java

Java es un lenguaje de programación diseñado bajo una arquitectura versátil y robusta. Basándonos en los fundamentos técnicos, sus características principales son:

- **Orientado a objetos:** Basado en el paradigma de objetos para la estructuración lógica del software.
- **Simple y seguro:** Proporciona la funcionalidad de un lenguaje potente eliminando las características más confusas de estos.
- **Interpretado:** Requiere de un sistema de ejecución para procesar el código.
- **Arquitectura independiente:** Java compila su código en un fichero objeto independiente de la máquina. Cualquier equipo que disponga del **JRE (Java Runtime Environment)** —entendido como el **sistema de ejecución** necesario para procesar los *bytecodes*— puede ejecutar dicho código.
- **Programas distribuidos y multitarea:** Proporciona herramientas nativas para gestionar procesos simultáneos y entornos de red.
- **El lenguaje más difundido:** Posee la mayor penetración en el mercado de desarrollo actual.
- **Plataformas compatibles:**
 - **Java SE (Standard Edition):** Plataforma estándar para aplicaciones de escritorio y desarrollo general.
 - **Java EE (Enterprise Edition):** Orientada a aplicaciones empresariales, corporativas y web.
 - **Java ME (Micro Edition):** Optimizada para dispositivos móviles y sistemas embebidos.

2. Sintaxis Básica: Identificadores y Comentarios

La construcción de un programa en Java exige el cumplimiento de normas sintácticas precisas para la definición de elementos:

Identificadores

Son los nombres asignados a variables, constantes y clases. Sus reglas son:

1. Deben comenzar obligatoriamente por una **letra**, un subrayado (`_`) o el signo de dólar (`$`).
2. Los caracteres subsiguientes pueden ser letras o dígitos.
3. Java es **case-sensitive** (distingue estrictamente entre mayúsculas y minúsculas).
4. No pueden coincidir con las **palabras clave** (keywords) reservadas del lenguaje.

Comentarios

Fundamentales para la documentación técnica, son ignorados por el compilador:

// Comentario de línea: Útil para anotaciones breves.

/* Comentario de bloque:

Utilizado para descripciones extensas
que ocupan varias líneas. */

3. Tipos de Datos Primitivos y Conversión

Java es un lenguaje **fuertemente tipado**, lo que obliga a definir el tipo de cada variable para determinar qué valores puede almacenar.

Tabla Comparativa de Tipos de Datos

Nombre	Tamaño (bits)	Rango / Descripción
byte	8	Enteros desde -128 hasta 127
short	16	Enteros desde -32,768 hasta 32,767
int	32	Enteros desde -2,147,483,648 hasta 2,147,483,647
long	64	Enteros desde -9,223,372,036,854,775,808 hasta 9,223,372,036,854,775,807
float	32	Coma flotante (reales) con 6 a 7 decimales
double	64	Coma flotante de doble precisión (hasta 15 decimales)
char	16	Caracteres basados en el código ASCII
boolean	-	Tipo lógico con valores true y false
void	-	Tipo especial que denota "vacío"

Jerarquía y Conversión (Casting)

La jerarquía de tipos numéricos sigue el orden: **double > float > int**.

1. **Conversión Implícita:** El compilador la realiza automáticamente cuando se asigna un tipo de menor rango a uno de mayor.

- *Ejemplo ASCII:* Una variable `int` puede asignarse a un `char` y viceversa (65 = 'A'; 'Z' = 90).
- 2. **Conversión Explícita (Casting):** Necesaria para transformar un tipo de mayor jerarquía a uno de menor. Sintaxis: `variable1 = (tipo) variable2;`
- 3. **Comportamiento en División Entera:** La división entre dos `int` resulta siempre en un entero truncado.
 - `x = (double) (10 / 4);` -> Resultado: **2.0** (la división ocurre antes del casting).
 - `x = (double) 10 / 4;` -> Resultado: **2.5** (el 10 se trata como double antes de la operación).

4. Operadores, Asignación y E/S Básica

Operadores Disponibles

- **Aritméticos:** `+`, `-`, `*`, `/`, `%` (resto), `Math.pow(a, b)` (potencia), `Math.sqrt(a)` (raíz).
- **Asignación y Cambio:** `=` (asignación), `++` (incremento), `--` (decremento).
- **Relacionales:** `<`, `>`, `<=`, `>=`, `==` (igualdad), `!=` (desigualdad).
- **Lógicos:** `&&` (AND), `||` (OR), `!` (NOT).

Sintaxis de Entrada/Salida (E/S)

Salida de datos:

```
System.out.print("Cadena");           // Sin salto de línea.
System.out.println("Cadena");         // Con salto de línea automático.
System.out.printf("Saldo: %.2f \n", s); // Formato: 2 decimales y salto de línea (\n).
```

Entrada de datos (Objeto Scanner): Requiere `import java.util.*;`. Para un uso correcto en arquitectura de consola, el objeto debe declararse **en la línea justo anterior al main**: `private static Scanner teclado = new Scanner(System.in);`

- **Leer Cadena de caracteres (String):** `teclado.nextLine();`
- **Leer int:** `Integer.parseInt(teclado.nextLine());`
- **Leer double:** `Double.parseDouble(teclado.nextLine());`
- **Leer char:** `teclado.nextLine().charAt(0);`

5. Estructuras de Control de Flujo

Alternativa Simple

Pseudocódigo	Java
--------------	------

SI condicion	if (condicion) {
instrucciones	instrucciones;
FIN SI	}

Alternativa Doble

Pseudocódigo	Java
SI condicion ... SI NO ... FIN SI	if (condicion) { ... } else { ... }

Alternativa Múltiple

Pseudocódigo	Java
SEGUN_VALOR expr	switch (expr) {
valor1: Bloque 1	case valor1: Bloque 1; break;
OTROS: Bloque Otros	default: Bloque Otros;
FIN SEGUN_VALOR	}

Bucle Mientras (while)

Pseudocódigo	Java
MIENTRAS condicion	while (condicion) {
Bloque instrucciones	Bloque instrucciones;
FIN MIENTRAS	}

Bucle Repetir (do-while)

Pseudocódigo	Java
REPETIR Bloque	do {
MIENTRAS condicion	Bloque; } while (condicion);

Bucle Para (for)

Pseudocódigo	Java
--------------	------

PARA v DE v_i A v_f CON INC= n	<code>for (inicializ; condicion; incremento)</code> <code>{</code>
Bloque instrucciones	<code>Bloque instrucciones; }</code>

Lógica de parámetros **for**:

1. **Inicialización**: Valor inicial de la variable de control (ej. $i = 1$).
2. **Condición**: Evaluación lógica para continuar el bucle (ej. $i \leq 5$).
3. **Incremento**: Evolución de la variable en cada iteración (ej. $i = i + 1$ o $i++$).

6. Arrays (Vectores) y Matrices (Bidimensionales)

Estructuras de datos que almacenan múltiples elementos del mismo tipo.

Configuración de Matrices (Caso: Tablero N x N)

- **Declaración**: En matrices cuadradas destinadas a juegos de parejas, el tamaño **N** debe ser par.
- **Acceso**: Se emplean dos índices de coordenadas: `[fila][columna]`.
- **Contenido**: Almacenamiento de parejas de números enteros.
- En Java, todos los índices de arrays y matrices **comienzan obligatoriamente en 0**.
- La posición inicial de cualquier matriz es `[0][0]`.
- Si una dimensión tiene tamaño N, el índice máximo permitido es **N-1**.

7. Anexo: Lógica de Implementación (Caso Práctico)

Flujo de trabajo para un turno en el juego de matrices:

1. **Visualización**: El sistema renderiza el estado actual del tablero.
2. **Selección**: El usuario introduce coordenadas para dos casillas.
3. **Validación de Igualdad de literales o valores**:
 - **Si los valores coinciden**: Acierto. Las casillas quedan destapadas permanentemente. No se consume intento.
 - **Si los valores difieren**: Fallo. Las casillas vuelven a su estado oculto y **se resta 1 intento**.
4. **Resolución de Partida**:
 - **Victoria**: Todas las parejas han sido halladas. Mostrar mensaje de felicitación.
 - **Derrota**: Los intentos se agotan (llegan a 0). Se muestra mensaje de fin de juego y la solución completa del tablero.